Engram

DeepSeek_Conditional Memory via Scalable Lookup.pdf（783 kB）

Architecture

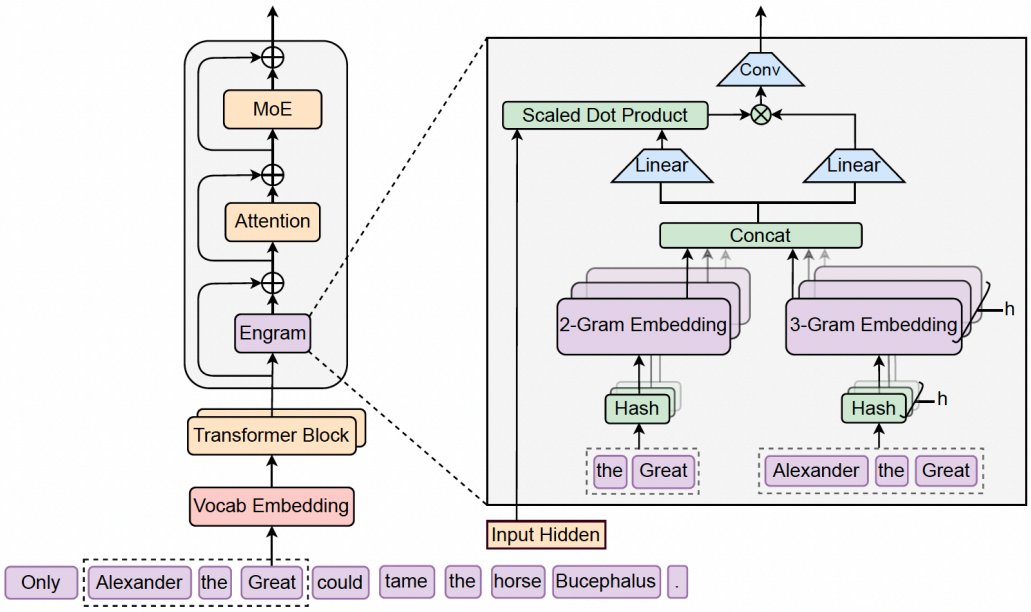


Figure 1 | **The Engram Architecture.** The module augments the backbone by retrieving static N-gram memory and fusing it with dynamic hidden states via context-aware gating. This module is applied only to specific layers to decouple memory from compute, leaving the standard input embedding and un-embedding module intact.

- Motivation: MoE via conditional computation方法虽然可以scale大模型的能力， 但由于transformer架构缺少对knowledge lookup天然的构造，所以在simulate retrievel时只能通过计算来进行，比较没效率。Engram的motivation就是为了解决这个问题
- Engram: 把conditonal memory作为除了conditional computation以外的一种补充的sparsity axis – a module that modernizes classic N–gram embedding for O(1) lookup，并且把这个module嵌入到了transformer中

Engram

Sparse Retrieval via Hashed N–grams

- local context –> static embedding vectors / memory via hashing

425字

Tokenizer Compression

Rank	Merge Count	Normalized Token	Original Tokens
1	163	’␣’	’\t’,’\n’,’\r’,’␣’,’␣␣’,’\n\n’,’␣␣␣’,’␣\n’,...
2	54	’a’	’A’,’a’,’␣a’,’␣A’,’á’,’ä’,’ã’,’ą’,’␣à’,’␣å’,’â’,...
3	40	’o’	’O’,’o’,’␣o’,’␣O’,’ó’,’ö’,’ô’,’õ’,’ő’,’ò’,...
4	35	’e’	’E’,’e’,’␣e’,’␣E’,’é’,’è’,’␣é’,’ę’,’ě’,’ê’,...
5	30	’i’	’I’,’i’,’␣I’,’␣i’,’í’,’ì’,’î’,’ī’,’ï’,...

Table 6 | The table illustrates Top-5 merged tokens by *Tokenizer Compression* and the overall compression ratio is 23.43% for our 128k tokenizer.

- 23% reduction for 128k tokenizer

Multi-Head Hashing

- 多头hashing是为了mitigate collision

Multi-Head Hashing. Directly parameterizing the combinatorial space of all possible N-grams is intractable. Following Tito Svenstrup et al. (2017), we adopt a hashing-based approach. To mitigate collisions, we employ K distinct hash heads for each N-gram order n. Each head k maps the compressed context to an index within an embedding table $\mathbf{E}_{n,k}$ (of prime size $M_{n,k}$) via a deterministic function $\varphi_{n,k}$:

$$z_{t,n,k} \triangleq \varphi_{n,k}(g_{t,n}), \quad \mathbf{e}_{t,n,k} = \mathbf{E}_{n,k}[z_{t,n,k}]. \tag{1}$$

In practice, $\varphi_{n,k}$ is implemented as a lightweight multiplicative-XOR hash. We construct the final memory vector $\mathbf{e}_t \in \mathbb{R}^{d_{\text{mem}}}$ by concatenating all retrieved embeddings:

$$\mathbf{e}_t \triangleq \mathop{\Big\|}_{n=2}^{N} \mathop{\Big\|}_{k=1}^{K} \mathbf{e}_{t,n,k}. \tag{2}$$

Context-aware Gating

- N-gram embedding本身没有上下文信息，而且可能因为hash collison带来一些noise，为了解决这个问题，这里用了一种类似attention的机制

425字

et al., 2017). Specifically, we utilize the current hidden state $\mathbf{h}_t$—which has aggregated global context via preceding attention layers—as a dynamic Query, while the retrieved memory $\mathbf{e}_t$ serves as the source for both Key and Value projections:

$$\mathbf{k}_t = \mathbf{W}_K \mathbf{e}_t, \quad \mathbf{v}_t = \mathbf{W}_V \mathbf{e}_t \tag{3}$$

where $\mathbf{W}_K, \mathbf{W}_V$ are learnable projection matrices. To ensure gradient stability (Dehghani et al., 2023), we apply RMSNorm (Zhang and Sennrich, 2019) to the Query and Key before computing the scalar gate $\alpha_t \in (0, 1)$:

$$\alpha_t = \sigma\left(\frac{\mathrm{RMSNorm}(\mathbf{h}_t)^\top \mathrm{RMSNorm}(\mathbf{k}_t)}{\sqrt{d}}\right). \tag{4}$$

The gated output is defined as $\tilde{\mathbf{v}}_t = \alpha_t \cdot \mathbf{v}_t$. This design enforces semantic alignment: if the retrieved memory $\mathbf{e}_t$ contradicts the current context $\mathbf{h}_t$, the gate α_t tends toward zero, effectively suppressing the noise.

Finally, to expand the receptive field and enhance the model's non-linearity, we introduce a short, depthwise causal convolution (Gu et al., 2022; Peng et al., 2023). Let $\tilde{\mathbf{V}} \in \mathbb{R}^{T \times d}$ denote the sequence of gated values. Using a kernel size w (set to 4), dilation δ (set to the max N-gram order) and SiLU activation (Elfwing et al., 2018), the final output $\mathbf{Y}$ is computed as:

$$\mathbf{Y} = \mathrm{SiLU}\left(\mathrm{Conv1D}(\mathrm{RMSNorm}(\tilde{\mathbf{V}}))\right) + \tilde{\mathbf{V}}, \tag{5}$$

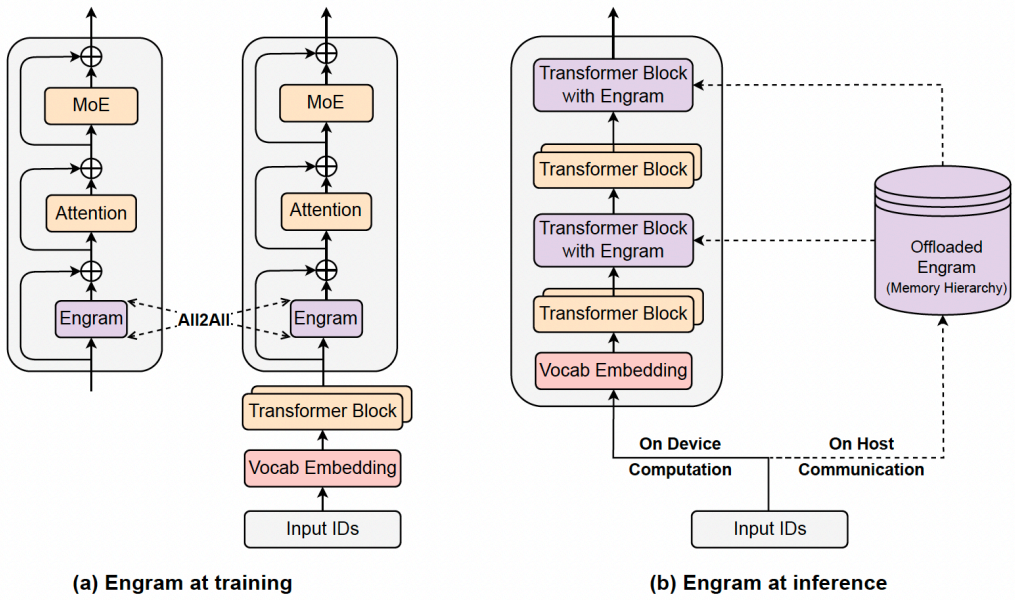
Integration w Multi-branch

- 大概的motivation是说：residual stream into M parallel branches对提高模型能力有好处 – 这里的M是mHC里的residual heads
- Specifically，embedding table e_t 和 W_V are shared across all M branches，W_K is distinct for each branch
- This design allows the linear projections (one W_V and M distinct W_K^m) to be fused into a single dense FP8 matrix multiplication, GPU utitilization up **(这里multi-branch的好处不是很懂。。)**
- **standard residual connection –> mHC,**

System Implementation & Efficienty

- "Scaling memory-augmented models is often constrained by the limited capacity of GPU high bandwidth memory (HBM). However, the deterministic retrieval mechanism of Engram naturally supports the decoupling of parameter storage from computational resources. Unlike MoE, which relies on runtime hidden states for dynamic routing, Engram's retrieval indices depend solely on the input token sequence."
- In training: sharding the tables across available GPU; In inference: memory is offloaded to host memory

425字



Sparsity allocation

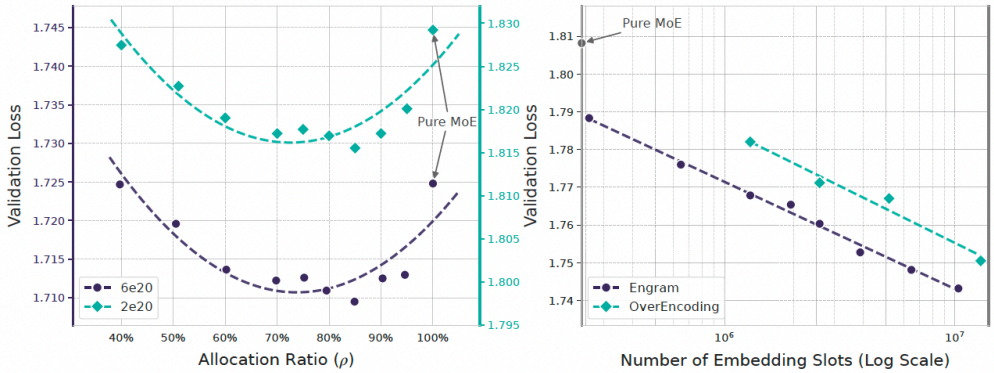


Figure 3 | **Sparsity allocation and Engram scaling.** Left: Validation loss across allocation ratios ρ. Two compute budgets are shown (2e20 and 6e20 FLOPs). Both regimes exhibit a U-shape, with hybrid allocation surpassing Pure MoE. Right: Scaling behavior in the infinite-memory regime. Validation loss exhibits a log-linear trend with respect to the number of embeddings.

Prev Work

- N-grammer:
  N-Grammer- Augmenting Transformers with latent n-grams.pdf (405 kB) - hash retrievel method

425字

- memory layer相关：Scaling Embedding Layers in Language Models <https://arxiv.org/abs/2502.01637> – 提取f-gram->embedding transformer->作为extra layer插入到standard transformer中
- bitwise XOR operator example

Python Example

In Python, the symbol for Bitwise XOR is `^`. Let's look at what happens when you XOR the numbers **5** and **3**.

1. **Convert to Binary**:
   - **5** in binary is `0101`
   - **3** in binary is `0011`
2. **Apply XOR bit by bit**:

Plaintext

```
  0 1 0 1  (5)
^ 0 0 1 1  (3)
-----------
  0 1 1 0  (Result)
```

3. **Convert back to Decimal**:
   - `0110` in binary is **6**.

425字